ACE Startup Infrastructure & CUDA Bridge Wiring

Status: Verified Operational
Verified date: 2026-05-15
Stack: SvelteKit 2 + N-API bridge using LibTorch / TensorRT-style scoring + simdjson + Port `8101` Topology Server

Recommended location:

```
sveltekit-frontend/docs/ACE_STARTUP_CUDA_BRIDGE.md
```

Optional short link from root docs:

```
README.md → docs/ACE_STARTUP_CUDA_BRIDGE.md
CLAUDE.md → ACE startup / native bridge safety notes
```

Purpose

This document records the operational wiring for the ACE startup path, the LibTorch CUDA bridge, the topology search server, and the native JSON parsing path. It is intended to prevent regressions where a startup smoke test passes superficially while the native bridge, topology router, or inference logging path silently fails.

The key principle is:

> ACE should fail open to safe CPU or fallback logic when native GPU helpers are unavailable, but diagnostics must clearly report which path was used.

Architecture Summary

```
SvelteKit 2 server runtime
  ↓
src/lib/server/env.server.ts
  ↓
ACE / KAG / topology routing services
  ↓
Topology Search Server :8101
  ↓
```

```
Optional native acceleration
  ├─ LibTorch / TensorRT-style N-API bridge
  ├─ simdjson fast JSON parsing
  └─ CPU fallback path
  ↓
Inference logging / diagnostics
  ├─ CouchDB :5984
  ├─ Postgres proxy :5434
  └─ TurboQuant / llama-server :8090 optional GPU boost
```

LibTorch CUDA Bridge: Path Regression Trap

The LibTorch reranker uses a native N-API bridge named:

```
tensorrt_bridge.node
```

This bridge provides fast GPU-side attention scoring / reranking support. Because the server-side AI services are deeply nested, pathing to this binary is a frequent point of failure.

Regression Fixed

Affected file:

```
src/lib/server/ai/libtorch-reranker.ts
```

Problem:
The file previously used `../../../../` to resolve the project root.

Root cause:
From:

```
src/lib/server/ai/
```

four `..` segments only reach:

```
sveltekit-frontend/
```

However, the native bridge lives outside the frontend folder at the monorepo root:

```
simd-bridge/
```

Fix:
The path was upgraded to five levels:

```
../../../../../simd-bridge/...
```

This reaches the monorepo root before entering `simd-bridge`.

Recommended Future Wiring

Prefer `process.cwd()` for service-local absolute resolution

For scripts or services that run from the `sveltekit-frontend` directory, prefer:

```
path.resolve(process.cwd(), '../simd-bridge/cpp/build/Release/
tensorrt_bridge.node')
```

instead of file-relative paths such as:

```
path.resolve(__dirname, '../../../../simd-bridge/...')
```

Reason: `process.cwd()` is more robust for service entrypoints that are always launched from `sveltekit-frontend`, while relative `require()` paths are fragile because they depend on the source file location.

Use silent fallback with visible warnings

Native bridge loading should never crash the whole ACE server path unless the current command is an explicit native verification command.

Recommended pattern:

```
let nativeBridge: unknown | null = null;
let nativeBridgeMode: 'native' | 'fallback' = 'fallback';

try {
  const bridgePath = path.resolve(
    process.cwd(),
    '../simd-bridge/cpp/build/Release/tensorrt_bridge.node'
```

```
  );

  nativeBridge = require(bridgePath);
  nativeBridgeMode = 'native';
} catch (err) {
  console.warn(
    '[libtorch-reranker] native bridge unavailable; falling back to CPU path',
    err instanceof Error ? err.message : err
  );
}
```

Rules:

- Use `console.warn`, not `console.error`, for optional native fallback.
- Do not crash the process when the CPU fallback is valid.
- Do crash only in explicit verification scripts such as `scratch/verify-libtorch.mjs`.
- Surface the selected mode in diagnostics.

Add diagnostic metadata

Use existing inference logging or diagnostics paths to report whether the runtime selected:

```
native
```

or:

```
fallback
```

Recommended diagnostic fields:

```
{
  bridge: 'libtorch-reranker',
  bridgeMode: 'native' | 'fallback',
  bridgePath,
  nativeAddonLoaded: boolean,
  simdjsonAvailable: boolean,
  cudaAvailable: boolean | 'unknown'
}
```

Good places to surface this:

```
logInference
ollama-diag
```

```
startup health check
smoke:hypergraph-routing
```

---

gRPC vs JSON vs N-API TypedArrays

The system uses multiple high-performance transport and serialization paths. These must not be confused.

Path	Transport	Serialization	Native Bridge Role
gRPC	HTTP/2	Protobuf binary	None; handled by `@grpc/grpc-js`
Ollama / TurboQuant-compatible APIs	HTTP/1.1	JSON text	`simdjson` fast validation / parsing
LibTorch reranker	N-API	TypedArrays / binary memory	Tensor scoring / attention reranking

Avoidance rules

Do **not** use `fastJsonParse` on:

```
gRPC streams
Protobuf buffers
TypedArray tensor payloads
```

Do use `fastJsonParse` for:

```
large Ollama / TurboQuant JSON synthesis responses, especially > 1 KB
```

This reduces V8 JSON parsing overhead and garbage collection pressure for large text responses.

Native memory warning

Both LibTorch bridge functions and simdjson helpers may be exposed through the same `.node` addon. If that addon fails to load, both native acceleration paths are unavailable.

Expected artifact location:

```
simd-bridge/cpp/build/Release/tensorrt_bridge.node
```

If this file is missing, ACE must continue through fallback logic, but the diagnostics should clearly say that native acceleration is disabled.

Startup Failure Prevention Checklist

Environment initialization

`src/lib/server/env.server.ts` must call dotenv setup before reading service URLs, database credentials, or inference settings.

Recommended first action:

```
import dotenv from 'dotenv';
dotenv.config();
```

Reason: standalone scripts launched with `npx tsx`, `node`, or smoke commands may not inherit the same environment-loading behavior as the SvelteKit dev server.

If `dotenv.config()` is missing or delayed, common failures include:

```
missing DB password
missing DATABASE_URL
wrong Postgres port
missing CouchDB URL
missing TurboQuant / Ollama URL
```

Service Sequencing

Startup should validate in this order:

```
L1: Start topology-search-server.mjs on :8101
L2: Verify LibTorch bridge load or fallback mode
L3: Confirm CouchDB inference log connectivity
L4: Confirm Postgres proxy connectivity
L5: Confirm optional TurboQuant / llama-server connectivity
```

The topology search server is required for routing. TurboQuant is optional GPU boost and should not block baseline ACE routing unless the specific test requires it.

Port Checklist

Port	Service	Required	Notes
`5434`	Postgres proxy	Yes	Required for database-backed ACE / atlas / graphify paths
`8101`	Topology Search Server	Yes	Required for routing
`5984`	CouchDB	Yes	Required for inference logs
`8090`	TurboQuant / llama-server	Optional	GPU boost / local model endpoint
`8788`	TRACE MCP Server	Context-dependent	Required for TRACE MCP tool calls

Verification Commands

Run from:

```
C:\Users\james\Videos\deeds-web-app\sveltekit-frontend
```

Verify LibTorch bridge

```
npx tsx scratch/verify-libtorch.mjs
```

Expected result:

```
native bridge loaded
```

or a clear fallback/diagnostic message if the bridge is intentionally unavailable.

Verify topology search

```
node scripts/ensure-search-engine.mjs --spawn
```

Expected result:

```
Topology Search Server ready on :8101
```

Verify hypergraph routing

```
npm run smoke:hypergraph-routing
```

Expected result:

```
PASS
```

Verify graphify daily path

```
npm run graphify:daily
```

If this fails with:

```
ECONNREFUSED 127.0.0.1:5434
```

then Postgres proxy is not listening on `5434`. Check Docker port mappings before debugging ACE routing.

PowerShell Startup Guardrails

Avoid Bash-style assignments in PowerShell scripts.

Incorrect:

```
FOO=bar
= "value"
```

Correct:

```
$env:FOO = "bar"
$FOO = "bar"
```

If startup prints:

```
=: The term '=' is not recognized as a name of a cmdlet, function, script file,
or executable program.
```

search the startup scripts with:

```
Select-String -Path .\*.ps1,.\scripts\*.ps1,.\package.json -Pattern "^\s*=|[A-
Z_]+="
```

Recommended Code Comments

Add near the native bridge resolver in `src/lib/server/ai/libtorch-reranker.ts`:

```
// IMPORTANT: this file lives at src/lib/server/ai/.
// The native bridge lives outside sveltekit-frontend at ../simd-bridge/.
// Prefer process.cwd() when this service is launched from sveltekit-frontend.
// Avoid fragile ../../../../ paths; four levels only reaches sveltekit-
frontend.
// Native bridge failure must fall back to CPU scoring unless this is an
explicit verification command.
```

Add near fast JSON parsing helpers:

```
// simdjson is only for JSON text payloads such as Ollama / TurboQuant
responses.
// Do not use fastJsonParse on gRPC streams, Protobuf buffers, or tensor
TypedArrays.
```

Add near startup health checks:

```
// Startup order matters:
// 1. Load env with dotenv.
// 2. Start topology search on :8101.
// 3. Detect native LibTorch/simdjson bridge mode.
// 4. Confirm CouchDB inference logs.
// 5. Confirm Postgres proxy on :5434.
// 6. Treat TurboQuant :8090 as optional unless the command explicitly requires
local GPU inference.
```

Operational Rule

Do not treat `graphify:daily complete` as proof that the full graphify chain succeeded. Some launchers may print completion after an inner command fails.

Trust the actual command exit code and the earliest failure in the log.

Common real blocker:

```
Error: connect ECONNREFUSED 127.0.0.1:5434
```

This is a Postgres availability or port-mapping issue, not a TRACE MCP or CUDA bridge issue.

Agentic Startup Guardrails

Why this broke

The observed failure was caused by several independent startup assumptions being allowed to pass as if they were one healthy system:

1. TRACE MCP started successfully on `:8788`, so the log looked healthy.
2. PowerShell encountered a malformed assignment and printed `=: The term '=' is not recognized...`, but startup continued.
3. `graphify:daily` launched and reached `atlas:build`.
4. `atlas:build` tried to connect to Postgres on `127.0.0.1:5434`.
5. Nothing was listening on `5434`, so `pg-pool` threw `ECONNREFUSED`.
6. The outer launcher still printed `graphify:daily complete`, even though the inner command failed.

This means the failure was not primarily caused by Gemma, Ollama, Hermes, TRACE MCP, CUDA, LibTorch, or the topology server. The first hard failure was database availability on the expected Postgres proxy port.

The deeper design issue is that the startup flow is currently too optimistic. It starts services and then runs graph/index jobs without a strict dependency gate that proves required ports, env variables, native bridge mode, and database connections are valid before expensive or agentic work begins.

Best Fix: Detached Startup with Strict Readiness Gates

Use a detached startup supervisor that separates service boot from graph/index work.

Recommended startup lanes:

```
Lane A: Required infrastructure
  ├─ Postgres proxy :5434
  ├─ CouchDB :5984
  ├─ Topology search :8101
  └─ TRACE MCP :8788 if MCP tools are enabled

Lane B: Optional acceleration
  ├─ LibTorch / CUDA native bridge
  ├─ simdjson native parser
  └─ TurboQuant / llama-server / Ollama :8090

Lane C: Validation smoke tests
  ├─ env.server.ts loads dotenv
  ├─ ports respond
  ├─ DB query succeeds
  ├─ native bridge reports native/fallback
  └─ topology route smoke passes

Lane D: Agentic repair / Gemma4 planning
  ├─ read-only diagnosis first
  ├─ propose fix
  ├─ run narrow verification command
  └─ never mutate infra without explicit command boundary
```

The key rule:

> Gemma4 should not be the first thing trying to discover broken infrastructure. Startup health should produce a machine-readable status file first, then Gemma4 can analyze that status and propose or run bounded fixes.

Detached Startup Contract

Create a startup status artifact that every agent and script can read:

```
.tmp/ace-startup-status.json
```

Example schema:

```
{
  "timestamp": "2026-05-15T00:00:00.000Z",
  "cwd": "C:/Users/james/Videos/deeds-web-app/sveltekit-frontend",
  "required": {
```

```
    "postgres5434": { "ok": false, "host": "127.0.0.1", "port": 5434, "error":
"ECONNREFUSED" },
    "couchdb5984": { "ok": true, "host": "127.0.0.1", "port": 5984 },
    "topology8101": { "ok": true, "host": "127.0.0.1", "port": 8101 }
  },
  "optional": {
    "turboquant8090": { "ok": false, "required": false },
    "libtorchBridge": { "ok": true, "mode": "native" },
    "simdjson": { "ok": true, "mode": "native" }
  },
  "decision": "BLOCK_GRAPHIFY",
  "nextSafeCommand": "docker compose up -d postgres"
}
```

Agentic tools should read this file before attempting graphify, indexing, or deep audit commands.

Suggested Script: `scripts/ace-startup-health.mjs`

```
import fs from 'node:fs';
import net from 'node:net';
import path from 'node:path';

const outPath = path.resolve(process.cwd(), '.tmp/ace-startup-status.json');
fs.mkdirSync(path.dirname(outPath), { recursive: true });

function checkPort(name, host, port, required = true, timeoutMs = 750) {
  return new Promise((resolve) => {
    const socket = new net.Socket();
    let done = false;

    const finish = (result) => {
      if (done) return;
      done = true;
      socket.destroy();
      resolve({ name, host, port, required, ...result });
    };

    socket.setTimeout(timeoutMs);
    socket.once('connect', () => finish({ ok: true }));
    socket.once('timeout', () => finish({ ok: false, error: 'TIMEOUT' }));
    socket.once('error', (err) => finish({ ok: false, error: err.code ||
err.message }));
    socket.connect(port, host);
  });
}
```

```
async function main() {
  const checks = await Promise.all([
    checkPort('postgres5434', '127.0.0.1', 5434, true),
    checkPort('couchdb5984', '127.0.0.1', 5984, true),
    checkPort('topology8101', '127.0.0.1', 8101, true),
    checkPort('traceMcp8788', '127.0.0.1', 8788, false),
    checkPort('turboquant8090', '127.0.0.1', 8090, false)
  ]);

  const requiredFailed = checks.filter((c) => c.required && !c.ok);
  const status = {
    timestamp: new Date().toISOString(),
    cwd: process.cwd(),
    checks: Object.fromEntries(checks.map((c) => [c.name, c])),
    decision: requiredFailed.length ? 'BLOCK_GRAPHIFY' : 'ALLOW_GRAPHIFY',
    failedRequired: requiredFailed.map((c) => c.name),
    nextSafeCommand: requiredFailed.some((c) => c.name === 'postgres5434')
      ? 'docker compose up -d postgres'
      : null
  };

  fs.writeFileSync(outPath, JSON.stringify(status, null, 2));
  console.log(`[ace-startup-health] wrote ${outPath}`);
  console.log(`[ace-startup-health] decision=${status.decision}`);

  if (requiredFailed.length) {
    process.exitCode = 1;
  }
}

main().catch((err) => {
  console.error('[ace-startup-health] fatal', err);
  process.exit(1);
});
```

Suggested `package.json` Wiring

```
{
  "scripts": {
    "ace:health": "node scripts/ace-startup-health.mjs",
    "ace:startup": "npm run ace:health && node scripts/ensure-search-engine.mjs
--spawn",
    "graphify:daily:safe": "npm run ace:health && npm run graphify:daily",
```

```
    "agent:preflight": "npm run ace:health && npm run smoke:hypergraph-routing"
  }
}
```

Use `graphify:daily:safe` instead of calling `graphify:daily` directly from agentic workflows.

PowerShell Detached Startup Example

```
$ErrorActionPreference = "Stop"

Write-Host "── ACE Detached Startup ──"

# Start required detached services first.
# Adjust service names to match the actual compose file.
docker compose up -d postgres couchdb

# Start topology search without blocking the shell.
Start-Process powershell -ArgumentList @(
  "-NoExit",
  "-Command",
  "cd `"$PWD`"; node scripts/ensure-search-engine.mjs --spawn"
)

# Optional TRACE MCP lane.
Start-Process powershell -ArgumentList @(
  "-NoExit",
  "-Command",
  "cd `"$PWD`"; npm run trace:mcp"
)

# Give services a moment to bind, then run strict health.
npm run ace:health

if ($LASTEXITCODE -ne 0) {
  Write-Error "ACE startup blocked. Read .tmp/ace-startup-status.json before
running graphify."
  exit $LASTEXITCODE
}

npm run smoke:hypergraph-routing
Write-Host "── ACE startup ready ──"
```

Do not use Bash-style assignment in this file. Use `$env:NAME = "value"` for environment variables.

`AGENTS.md` / `LLMS.md` Master Guardrail

Add this to the root `AGENTS.md`, `LLMS.md`, or `CLAUDE.md` master file.

````
# ACE Agentic Workflow Guardrails

## Startup truth source

Before running graphify, indexing, deep audit, topology synthesis, or codebase
mutation, read:

```text

.tmp/ace-startup-status.json
````

If the file does not exist, run:

```
npm run ace:health
```

Hard blocks

Do not run graphify/indexing commands if startup status says:

```
BLOCK_GRAPHIFY
```

Common blockers:

- Postgres proxy `127.0.0.1:5434` unavailable
- CouchDB `127.0.0.1:5984` unavailable
- Topology search `127.0.0.1:8101` unavailable
- `.env` not loaded before server-side scripts

Required command order

```
npm run ace:health
node scripts/ensure-search-engine.mjs --spawn
npm run smoke:hypergraph-routing
npm run graphify:daily:safe
```
```

Native bridge rules

- Never assume CUDA / LibTorch is available.
- Detect `native` vs `fallback` mode.
- Missing `tensorrt_bridge.node` should not crash normal ACE startup.
- Explicit verification commands may fail hard.

Serialization rules

- gRPC uses HTTP/2 + Protobuf. Do not parse with simdjson.
- Ollama / TurboQuant APIs use HTTP JSON. Large JSON responses may use simdjson.
- LibTorch N-API uses TypedArrays / binary memory. Do not JSON-parse tensor payloads.

Agentic repair rules

Gemma4 / Ollama agents may:

- inspect logs
- read `.tmp/ace-startup-status.json`
- propose fixes
- run narrow smoke tests
- update docs
- create patches only when explicitly assigned

Gemma4 / Ollama agents must not:

- mutate Postgres, CouchDB, Redis, Qdrant, or Neo4j during startup repair
- run heavy GPU indexing automatically
- treat `graphify:daily complete` as proof of success
- ignore the first thrown error in a startup log
- rewrite native bridge paths without preserving fallback behavior

````
---

## Gemma4 / Ollama Agentic Error-Fixing Loop

Recommended loop:

```text
1. Human or script starts detached infrastructure.
2. ace-startup-health writes .tmp/ace-startup-status.json.
3. Gemma4 reads the status file and latest logs.
4. Gemma4 classifies the error:
   - env error
   - port unavailable
   - native bridge unavailable
````
````

```
   - serialization mismatch
   - graphify/indexing failure
5. Gemma4 proposes the smallest safe fix.
6. Human or bounded controller approves a command.
7. Run one narrow verification command.
8. Append result to startup diagnostics.
```

This keeps Gemma4 useful without letting it become an uncontrolled startup mutator.

Best Immediate Fix for the Observed Failure

The observed hard blocker is:

```
ECONNREFUSED 127.0.0.1:5434
```

Run:

```
docker ps --format "table {{.Names}}    {{.Ports}}"
Test-NetConnection 127.0.0.1 -Port 5434
```

If `5434` is not open, either start the Postgres proxy/container:

```
docker compose up -d postgres
```

or update the app environment to the actual exposed port, commonly:

```
DATABASE_URL=postgres://USER:PASSWORD@127.0.0.1:5432/DBNAME
```

Then verify in this order:

```
npm run ace:health
npm run smoke:hypergraph-routing
npm run graphify:daily:safe
```